

Objektumorientált programozás

3. Beadandó

Dokumentáció

Ballai Péter Lóránt
EYSEED

2026.05.22

Tartalom

Feladatleírás.....	2
Fejlesztői dokumentáció.....	4
Típusleírás és Reprezentáció.....	4
Műveletek.....	6
Osztálydiagram.....	6
Implementáció.....	7
Terem osztály megvalósítása.....	7
Kis Terem osztály megvalósítása.....	7
Nagy Terem osztály megvalósítása.....	8
VIP Terem osztály megvalósítása.....	8
Terem Gyár osztály megvalósítása.....	8
Ülőhely osztály megvalósítása.....	9
Filmszínházkezelő osztály megvalósítása.....	11
Tesztelési terv.....	13
Szekvencia Diagram.....	14

Objektumelvű/Objektumorientált programozás

3. beadandó feladat

Feladatleírás

7. Modellezzük egy **filmszínház** jegyvásárlásait!

A filmszínház több vetítőteremben vetít a filmeket megadott időintervallumokban (előadások). A vetítőterem eltérő méretűek: széksoraikat betűk (A, B, C, ...), a sorok ülőhelyeit számok (1, 2, 3, ...) azonosítják. A nézők előre foglalhatnak helyet egy előadásra (terem és időintervallum), majd a jegyet megvásárolják. Foglалás nélkül is lehet szabad helyet vásárolni. Egy jegy árara (természetes szám) kedvezmény adható attól függően, hogy mi vetítőterem kategóriája (nagy, közepes, VIP) illetve, hogy a birtokosa (tehát nem az, aki megvette) felnőtt, diák, gyerek, nyugdíjas vagy törzstag: $\text{jegyár} \cdot ((100 - \text{kedvezmény})/100.0)$

kedvezmény	gyerek	diák	felnőtt	nyugdíjas	törzstag
kis terem	40	30	10	30	30
nagy terem	40	20	0	20	30
VIP terem	0	0	0	0	0

- Melyik filmet nézte meg a legtöbb néző?
- Számoljuk meg egy adott előadásra megvett, csak lefoglalt, illetve szabad helyeket!

Készítsen használati eset diagramot! Adjon meg a fenti feladathoz egy olyan objektum diagramot, amely megmutatja egy filmszínház egy vetítőtermét, ahhoz rendelt két előadást, azt, hogy egyik előadásra két helyet már egy néző lefoglalt, egy harmadikat, egy gyerek számára valaki már kifizetett.

Rajzolja fel a feladat osztály diagramját! A triviális getter/setter-eket ne írja bele a modellbe: azoknak a privát/védett adattagoknak a láthatóságát, amelyekhez getter-t is, és setter-t is kell készíteni, jelölje publikusnak; a csak getter-rel rendelkező adattag mellé írjon „readonly”-t.

Készítse el egy előadás objektum állapotgépét! Ennek egy állapotát az ülőhelyeinek állapotai együttesen határozzák meg. Egy ülőhely állapota lehet „szabad”, „foglalt”, vagy „eladott”. Tervezze meg az állapot-átmeneteket megvalósító tevékenységeket, amelyeket majd az Előadás osztály metódusaiként definiálhat.

Egészítse ki az osztálydiagramot az objektum-kapcsolatokat létrehozó metódusokkal, valamint a feladat kérdéseit megválaszoló metódusokkal. A metódusok leírásában a félév első felében bevezetett végrehajtható specifikációs jelöléseket használja. Azoknak a konstruktoroknak a törzsét, amelyek kizárólag az adattagok inicializálását végzik, nem kell feltüntetni. Ilyenkor a konstruktor paraméterlistája helyén elég felsorolni az inicializálandó adattagok neveit. Az összes közvetlen (tehát nem szerepnév) adattag felsorolása helyett elég „...”-ot írni.

Használjon tervezési mintákat, és mutasson rá, hogy hol melyiket alkalmazta.

Implementálja a modellt! Szerkesszen olyan szöveges állományt, amelyből fel lehet populálni a modell alapján egy filmszínház termeit, előadásait, foglalásait és jegyvásárlásait! Készítsen tesztteseteket, néhánynak rajzolja fel a szekvencia diagramját, és hozzon létre ezek kipróbálására automatikusan tesztkörnyezetet!

Fejlesztői dokumentáció

Típusleírás és Reprezentáció

Terem (Venue) - Absztrakt

Típusleírás: Egy általános, absztrakt termet leíró típus, amely összefogja a közös tulajdonságokat.

Reprezentáció:

- Name (string): A terem neve.
- Category (VenueCategory): A terem kategóriája.
- RowCount (int): A sorok száma a teremben.
- SeatsPerRow (int): Az oszlopok száma a teremben.
- GetDiscount (ViewerType) : int : Egy absztrakt metódus, amivel kitudjuk számolni a kedvezmény értékét.

KisTerem (SmallVenue)

Típusleírás: A Kis típusú termet megvalósító osztály.

Reprezentáció: Örökli a Venue adattagjait.

NagyTerem (LargeVenue)

Típusleírás: A Nagy típusú termet megvalósító osztály.

Reprezentáció: Örökli a Venue adattagjait.

VipTerem (VipVenue)

Típusleírás: A VIP típusú termet megvalósító osztály.

Reprezentáció: Örökli a Venue adattagjait.

TeremGyár (VenueFactory) - Factory

Típusleírás: Egy gyártó osztály a termekhez.

Reprezentáció: CreateVenue(Venue) : A paraméternek megfelelően visszatér egy teremmel.

Ülés (Seat)

Típusleírás: Egy ülést ábrázoló osztály.

Reprezentáció:

- Row (char) : A szék sorát tárolja.
- Number (int) : A szék oszlopát tárolja.
- State (SeatState) : A szék állapotát tárolja.
- TransitionBooked(): Lefoglalja a székét.
- TransitionSold(): Véglegesen lefoglalja a székét, miután kifizették.

Előadás (Show)

Típusleírás: Egy előadást ábrázoló osztály.

Reprezentáció:

- Id (int) : Az előadás azonosítója.
- Name (string) : A film neve.
- Time (string) : Az előadás időpontja.
- ShowVenue(Venue): Az előadás terme.
- Seats(Seat*): Az előadáshoz tartozó székek.
- BasePrice(int): Az alapértelmezett ár.
- GetSeat(Seat) : Visszaadja a megadott sorú és oszlopú székét.
- GetFreeCount(int) : Visszaadja a szabad helyek számát.
- GetBookedCount(int) : Visszaadja a lefoglalt helyek számát.
- GetSoldCount(int) : Visszaadja a kifizetett helyek számát.
- CalculatePrice(double) : Kiszámolja mennyi a kedvezményes ár.

Műveletek

1. Fájl beolvasása és feldolgozása

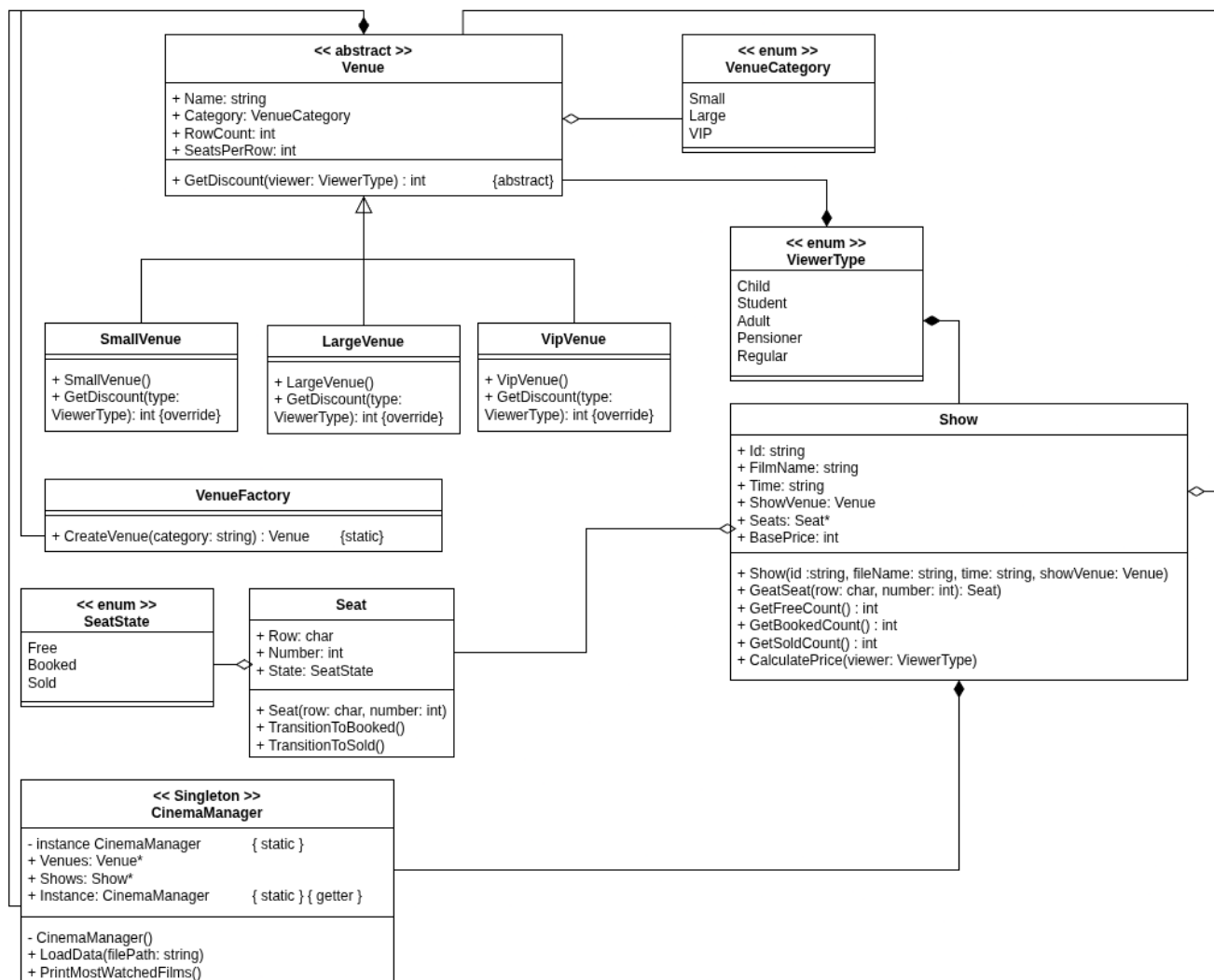
Fájl sorainak beolvasása, osztálpéldányok tárolása.

2. Feldolgozott adatok megjelenítése

A legtöbb nézővel rendelkező film kiírása.

3. Egy előadás statisztikája kiírása.

Osztálydiagram



Implementáció

Terem osztály megvalósítása

A terem absztrakt osztály konstruktora, amit az öröklött osztályok meghívnak példányosításkor.

```
Room(name: string, category: RoomCategory, rowCount: int,
seatsPerRow: int)
Name := name
Category := category
RowCount := rowCount
SeatsPerRow := seatsPerRow
```

Az osztály absztrakt módsere, amit a leszármazott osztályok egyedi kedvezménytáblázat alapján írnak felül:

```
abstract GetDiscount(viewer: ViewerType) : int
```

Kis Terem osztály megvalósítása

A kis terem osztály konstruktora és a hozzá tartozó kedvezmény-meghatározás.

```
SmallRoom(name: string, rowCount: int, seatsPerRow: int)
base(name, Small, rowCount, seatsPerRow)

GetDiscount(viewer: ViewerType) : int
if viewer = Child then return 40
elseif viewer = Student then return 30
elseif viewer = Adult then return 10
elseif viewer = Pensioner then return 30
elseif viewer = Regular then return 30
endif
return 0
```

Nagy Terem osztály megvalósítása

A nagy terem osztály konstruktora és a hozzá tartozó kedvezmény-meghatározás.

```
LargeRoom(name: string, rowCount: int, seatsPerRow: int)
base(name, Large, rowCount, seatsPerRow)

GetDiscount(viewer: ViewerType) : int
if viewer = Child then return 40
elseif viewer = Student then return 20
elseif viewer = Adult then return 0
elseif viewer = Pensioner then return 20
elseif viewer = Regular then return 30
endif
return 0
```

VIP Terem osztály megvalósítása

A VIP terem osztály konstruktora, ahol a szabályzat értelmében nincs kedvezmény.

```
VipRoom(name: string, rowCount: int, seatsPerRow: int)
base(name, VIP, rowCount, seatsPerRow)

GetDiscount(viewer: ViewerType) : int
return 0
```

Terem Gyár osztály megvalósítása

A Terem gyár (RoomFactory) osztály felelős a megfelelő teremobjektum példányosításáért a megadott kategória alapján.

```
CreateRoom(categoryType: string) : Room
if categoryType = "Small" then return new SmallRoom()
elseif categoryType = "Large" then return new LargeRoom()
elseif categoryType = "VIP" then return new VipRoom()
endif
error
```


Ülőhely osztály megvalósítása

Az ülőhely osztály konstruktora és az állapotgép átmeneteit vezérlő tevékenységek.

```
Seat(row: char, number: int)
Row := row
Number := number
State := Free

TransitionToBooked()
if State = Free
then State := Booked
endif

TransitionToSold()
if State = Free or State = Booked
then State := Sold
endif
```

Előadás osztály megvalósítása

Az előadás osztály konstruktora, mely automatikusan legenerálja a hozzá rendelt terem széksorait, valamint a helyek statisztikáját lekérdező metódusok.

```

Show(id: string, filmName: string, time: string, showRoom: Room)
Id := id
FilmName := filmName
Time := time
ShowRoom := showRoom
BasePrice := 2000
Seats.Clear()

currentRow := 'A'
for i := 0 .. showRoom.RowCount - 1 loop
    for j := 1 .. showRoom.SeatsPerRow loop
        Seats.Add(new Seat(currentRow, j))
    endloop
    currentRow := currentRow + 1
endloop

GetSeat(row: char, number: int) : Seat
forall s in Seats loop
    if s.Row = row and s.Number = number
    then return s
    endif
endloop
return null

GetFreeCount() : int
count := 0
forall s in Seats loop
    if s.State = Free then count := count + 1 endif
endloop
return count

GetBookedCount() : int
count := 0
forall s in Seats loop
    if s.State = Booked then count := count + 1 endif
endloop
return count

GetSoldCount() : int
count := 0
forall s in Seats loop
    if s.State = Sold then count := count + 1 endif
endloop
return count

CalculatePrice(viewer: ViewerType) : double
discount := ShowRoom.GetDiscount(viewer)
return BasePrice * ((100.0 - discount) / 100.0)

```

Filmszínházkezelő osztály megvalósítása

A központi, singleton mintát megvalósító osztály adatbeolvasó és statisztikai kiértékelő metódusai.

```

LoadData(filePath: string)
sr := new StreamReader(filePath)
while (line := sr.ReadLine()) != null loop
    parts := line.Split(";")
    if parts[0] = "ROOM"
        then room := RoomFactory.CreateRoom(parts[2])
        room.Name := parts[1]
        room.RowCount := int.Parse(parts[3])
        room.SeatsPerRow := int.Parse(parts[4])
        Rooms.Add(room)
    elseif parts[0] = "SHOW"
        then assignedRoom := null
        forall r in Rooms
            loop
                if r.Name = parts[4]
                    then assignedRoom := r break
            endif
        endloop
        show := new Show(parts[1], parts[2], parts[3], assignedRoom)
        Shows.Add(show)
    elseif parts[0] = "BOOK"
        then targetShow := null
        forall s in Shows
            loop
                if s.Id = parts[1]
                    then targetShow := s
                    break
            endif
        endloop
        seat := targetShow.GetSeat(parts[2][0], int.Parse(parts[3]))
        if seat != null
            then seat.TransitionToBooked()
        endif
    elseif parts[0] = "BUY"
        then targetShow := null
        forall s in Shows
            loop
                if s.Id = parts[1]
                    then targetShow := s break
            endif
        endloop
        seat := targetShow.GetSeat(parts[2][0], int.Parse(parts[3]))
        if seat != null
            then seat.TransitionToSold()
        endif
    endif
endloop
sr.Close()
PrintMostWatchedFilm()
films.Clear()
counts.Clear()
forall s in Shows
    loop
        soldCount := s.GetSoldCount()
        found := false for i := 0 .. |films| - 1
            loop
                if films[i] = s.FilmName
                    then counts[i] := counts[i] + soldCount
                    found := true break
            endif
        endloop
        if not found
            then films.Add(s.FilmName)
            counts.Add(soldCount)
        endif
    endloop
max := -1
bestFilm := ""
for i := 0 .. |films| - 1
    loop
        if counts[i] > max
            then max := counts[i]
            bestFilm := films[i]
        endif
    endloop
print("Most Watched Film: " + bestFilm + " (" + max + " tickets sold)")

```

Tesztelési terv

1. Terem gyár (RoomFactory) példányosításának ellenőrzése helyes kategória azonosító ("Small", "Large", "VIP") esetén
2. Terem gyár (RoomFactory) kivételkezelésének ellenőrzése ismeretlen vagy hibás kategória azonosító megadása esetén
3. Ülőhely (Seat) állapotgépének ellenőrzése sikeres helyfoglalás (Free -> Booked állapotátmenet) esetén
4. Ülőhely (Seat) állapotgépének ellenőrzése sikeres jegyvásárlás (Free -> Sold, illetve Booked -> Sold állapotátmenet) esetén
5. Ülőhely (Seat) állapotgépének ellenőrzése érvénytelen állapotátmenet (például már eladott jegy ismételt lefoglalása vagy megvásárlása) kísérlete esetén
6. Terem (Room) típusfüggő kedvezményeinek (GetDiscount) ellenőrzése a normál (Kis és Nagy) termeknél, különböző nézőtípusok (Gyerek, Diák, Felnőtt, Nyugdíjas, Törzstag) esetén
7. VIP Terem kedvezmény-kalkulációjának ellenőrzése (annak megerősítése, hogy minden nézőtípus esetén szigorúan 0% a kedvezmény)
8. Előadás (Show) statisztikai metódusainak (GetFreeCount, GetBookedCount, GetSoldCount) ellenőrzése vegyes állapotú (szabad, foglalt, eladott) ülőhelyek megléte esetén
9. Előadás (Show) jegyár-kalkulációjának (CalculatePrice) ellenőrzése az alapár és a terem által biztosított kedvezmény helyes érvényesítése alapján
10. Filmszínház kezelő (CinemaManager) adatbeolvasásának (LoadData) ellenőrzése helyesen formázott szöveges állomány (termek, előadások, foglalások, vásárlások) esetén
11. Filmszínház kezelő (CinemaManager) statisztikájának ellenőrzése a legtöbb nézőt vonzó film (PrintMostWatchedFilm) helyes összesítése és kiválasztása során, több azonos című előadás adatainak összevonásával

Szekvencia Diagram

